

Homework 03: Grades and Grids

Two small programs. The first asks you to pick the right conditional for the job. The second shows you why labelled break exists by making you live without it first.

Part One: Grades

Create `Grades.java`. Given this list of scores:

```
int[] scores = {95, 83, 71, 64, 42, 100, 0};
```

Print each score with its letter grade and a short comment, then the class average with its grade.

Grade boundaries: 90+ is A, 80+ is B, 70+ is C, 60+ is D, anything less is F.

Target output:

```
95  A   excellent
83  B   solid
71  C   passing
64  D   scraped through
42  F   see me
100 A   excellent
0   F   see me
```

```
Class average: 65 (D)
```

The Decision You Have To Make

One of these two jobs suits a `switch` expression and the other does not.

- Turning a **score** into a letter
- Turning a **letter** into a comment

Work out which is which, use `switch` only where it fits, and leave a comment explaining why the other one is an if-chain. Forcing a switch where it does not belong is a worse answer than not using one at all.

Part Two: Find In Grid

Create `FindInGrid.java`. Search this grid for a target value and report its row and column, or say it is absent:

```
int[][] grid = {
    { 7, 12, 3},
    { 9, 5, 21},
    {14, 2, 18}
};
```

Write the search **twice**:

1. `withFlag` — using a `boolean found` variable and a plain `break`
2. `withLabel` — using a labelled break

Run both against `21`, which is present, and `99`, which is not.

Then write a comment comparing them. Specifically: list what the flag version needs that has nothing to do with searching a grid.

Acceptance Criteria

- ☐ Both files run with `java Grades.java` and `java FindInGrid.java`
- ☐ `Grades.java` uses a `switch` expression for exactly one of the two jobs
- ☐ A comment explains why the other job is not a switch
- ☐ Columns line up; `100` and `0` both render correctly
- ☐ `FindInGrid.java` contains both `withFlag` and `withLabel`
- ☐ Both versions produce identical output for `21` and for `99`
- ☐ A comment lists the bookkeeping the flag version needs

Stretch

There is a third way to escape both loops that needs neither a flag nor a label. Find it, implement it as `withExtractedMethod`, and say when you would prefer it over the labelled break.

Hint, if the switch will not compile

A switch expression has to produce a value for every possible input. Over an `int` or a `String` that means you cannot cover everything by listing cases, so the compiler insists on a `default`. Over an `enum` with every constant listed, it does not. That difference is the whole reason one of these two jobs fits a switch better than the other.